# Praktikum 6 — Abstraction

## A. Identitas dan Tujuan Praktikum

Modul ini melatih seluruh materi abstraction (abstraksi: menampilkan *apa* yang dilakukan sebuah objek dan menyembunyikan *bagaimana* caranya) dalam satu proyek toko online UMKM. Mahasiswa membangun proyek dari folder kosong dan dikerjakan di laboratorium selama 120 menit, lalu tugas dikerjakan di rumah.

| | |
|---|---|
| Mata kuliah | Praktikum Pemrograman Berorientasi Objek (1 SKS) |
| Topik | Abstraction: abstract class, interface, default/static/private method, interface inheritance, `final`, nested class |
| Bahasa dan alat | Java >21 (JDK >21), VS Code + Extension Pack for Java |
| Bentuk kerja | Individu |
| Latihan (L0–L5) | Di laboratorium, 120 menit, dikumpulkan di akhir sesi |
| Tugas (T1–T4) | Dikerjakan di rumah, deadline pengumpulan: **6x24 jam setelah pelaksanaan praktikum** |
| Bahan | Paket `Praktikum-Abstraction-Mahasiswa.zip` |

### Tujuan Praktikum

1. Menjelaskan konsep abstraction, termasuk perbedaan abstract class dan interface serta kapan masing-masing tepat dipakai.
2. Mengimplementasikan abstraction di Java: abstract class dan method, interface beserta default/static/private method, interface inheritance, final, serta static nested, inner, dan anonymous class.
3. Merancang program yang mudah diperluas dengan prinsip *program to an interface*, termasuk menambah fitur ke interface tanpa merusak kode lama dan menggambarkannya dalam class diagram UML.
4. Mengevaluasi desain kode: menemukan dan memperbaiki masalah abstraction pada kode yang ada, serta memberi alasan atas pilihan desain yang dibuat.
5. Membangun dan menguji proyek Java secara mandiri di VS Code, termasuk membaca pesan *error compiler* dan hasil pemeriksaan otomatis untuk memperbaiki kode sendiri.

### Persiapan Praktikum

1. Pastikan `java -version` dan `javac -version` menampilkan versi > 21.
2. Unduh dan ekstrak file `Praktikum-Abstraction-Mahasiswa.zip`. Zip ini berisi alat bantu yang akan disalin ke proyek baru mahasiswa di L0, bukan proyek yang dibuka langsung.
3. Baca ulang slide perkuliahan (Abstraction) dan Deitel ed. 11 subbab 10.4, 10.7, 10.9–10.11, 10.13.

### Cara memakai SelfCheck

`SelfCheck` adalah program pemeriksa otomatis yang menguji kode Anda tanpa perlu diubah. Jalankan kapan saja, pemeriksaan yang belum dikerjakan memang akan berstatus `[GAGAL]`.

**Cara 1 – Tombol Run di VS Code.** Buka file yang punya `main` (misalnya `src/selfcheck/SelfCheck.java`), lalu klik **Run** di atas `main`. VS Code otomatis mengompilasi seluruh proyek ke folder **`bin`**.

**Cara 2 – Terminal VS Code (wajib untuk program yang butuh argumen, seperti `NimParams`).** Buka **Terminal → New Terminal**; terminal ini otomatis berada di folder proyek. Karena VS Code sudah mengompilasi ke `bin`, cukup jalankan:

```
java -cp bin selfcheck.SelfCheck
java -cp bin shop.app.ShopApp
java -cp bin tools.NimParams 2401010057
java -cp bin selfcheck.TugasCheck
java -cp bin review.BadShop
```

**Cara 3 – Kompilasi manual tanpa VS Code.** Hasil kompilasi manual disimpan di folder `out`, jadi semua perintah `java` memakai `-cp out`:

- Windows (CMD), dari folder proyek: `dir /s /b src\*.java > sources.txt`, lalu `javac -d out @sources.txt`, lalu `java -cp out selfcheck.SelfCheck`.
- Linux/macOS: `javac -d out $(find src -name "*.java")`, lalu `java -cp out selfcheck.SelfCheck`.

Aturan pentingnya: folder setelah `-cp` harus folder tempat file `.class` berada, yaitu `bin` (VS Code) atau `out` (kompilasi manual). Jangan mencampur keduanya.

**Jika muncul `Error: Could not find or load main class ...`**

| Penyebab | Cara mengecek | Perbaikan |
|---|---|---|
| `-cp` menunjuk folder yang salah atau belum ada, misalnya `out` padahal proyek VS Code memakai `bin` | Lihat isi folder proyek: yang ada `bin` atau `out`? | Pakai `-cp bin` untuk proyek dari L0 |
| Terminal tidak berada di folder proyek | Ketik `dir` (Windows) atau `ls`; harus terlihat `src` dan `bin` | `cd` ke folder `abstraction-<NIM>` |
| Kompilasi gagal karena ada file lain yang error, sehingga class yang dicari tidak pernah dibuat | Ada baris `error:` saat `javac`, ada tanda merah di panel **Problems**, atau VS Code menampilkan *Build failed* | Perbaiki file yang disebut, atau pindahkan sementara file yang belum selesai keluar dari `src` |
| Nama class salah ketik | Harus lengkap dengan package dan huruf besar/kecil yang sama, misalnya `tools.NimParams` | Salin persis dari modul |

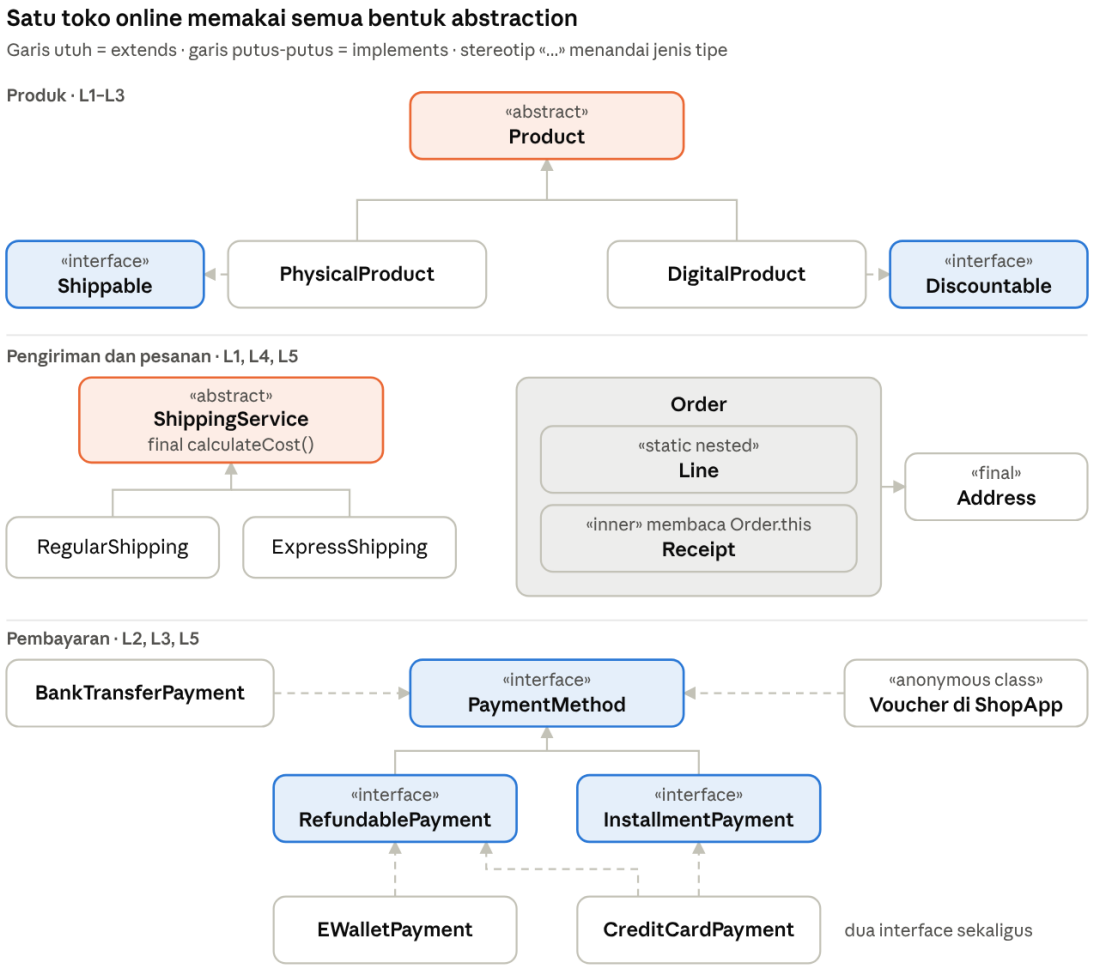
**Cara membaca hasil SelfCheck.** Setiap pemeriksaan yang gagal diberi alasan setelah tanda `->`:

| Alasan | Arti | Yang perlu dilakukan |
|---|---|---|
| `class/interface belum ada: ...` | File belum dibuat, atau nama/package-nya berbeda | Buat file dengan nama dan package yang persis sama |
| `method/konstruktor tidak ditemukan: ...` | Nama, tipe parameter, atau `public` tidak sesuai spesifikasi | Cocokkan method dengan modul |
| `hasil tidak sesuai spesifikasi` | Kode berjalan, tetapi hasil atau strukturnya salah (misalnya angka keliru atau lupa `final`) | Baca ulang spesifikasi butir tersebut |
| `error kode/exception ...` | Terjadi exception saat kode Anda dipanggil | Periksa validasi dan nilai `null` |

SelfCheck membantu, tetapi tidak menggantikan pemahaman. Nilai latihan juga memuat pemeriksaan manual oleh asisten praktikum/dosen.

## B. Peta Proyek Praktikum

Di akhir latihan, proyek mahasiswa berisi 21 tipe dalam tiga area: produk, pengiriman dan pesanan, serta pembayaran. Diagram ini adalah target akhir, kembalilah ke sini saat bingung di mana sebuah class ditempatkan.




class diagram akhir proyek · 3 area, 21 tipe

`CheckoutService` (package `shop.service`) tidak digambar agar diagram tetap terbaca. Class ini hanya bergantung pada abstraksi: `Product`, `Shippable`, `ShippingService`, `PaymentMethod`, dan `RefundablePayment`.

| Package | Isi |
|---|---|
| `shop.model` | `Product`, `PhysicalProduct`, `DigitalProduct`, `Address`, `Order` |
| `shop.shipping` | `Shippable`, `ShippingService`, `RegularShipping`, `ExpressShipping` |
| `shop.discount` | `Discountable` |
| `shop.payment` | `PaymentMethod`, `RefundablePayment`, `InstallmentPayment`, `EWalletPayment`, `BankTransferPayment`, `CreditCardPayment` |
| `shop.service` | `CheckoutService` |
| `shop.app` | `ShopApp` |

# C. Latihan Praktikum

Setiap latihan berisi analogi, spesifikasi, kode, kesalahan umum, dan checkpoint SelfCheck. Kode yang ditulis lengkap boleh diketik ulang. Bagian bertanda **TULIS SENDIRI** wajib mahasiswa kerjakan berdasarkan spesifikasi. Semua uang memakai tipe `long` dalam satuan rupiah.

### L0 – Membuat proyek baru dari awal (10 menit)

Mahasiswa membuat proyek Java sendiri di VS Code tanpa build tool (alat pengelola build seperti Maven atau Gradle). Zip `Praktikum-Abstraction-Mahasiswa.zip` **tidak dibuka sebagai proyek**, isinya hanya disalin sebagai alat bantu.

**Langkah 1 – Buat proyek.**

1. Buka VS Code. Tekan `Ctrl+Shift+P` (macOS: `Cmd+Shift+P`), ketik **Java: Create Java Project...**, lalu pilih **No build tools**.
2. Pilih folder induk tempat proyek disimpan, misalnya `D:\praktikum-oop\P6`.
3. Beri nama proyek `abstraction-<NIM>`, misalnya `abstraction-2401010057`. VS Code membuka proyek baru di jendela tersebut.

**Langkah 2 – Kenali isi proyek.** VS Code membuat struktur berikut:

```
abstraction-2401010057/
├── .vscode/settings.json   <- tells VS Code that the source folder is src
├── lib/                    <- external .jar files (not used today)
├── src/App.java            <- sample program
└── README.md
```

Folder `bin/` (hasil kompilasi `.class`) muncul otomatis setelah program pertama dijalankan.

**Langkah 3 – Pastikan JDK >= 21 terpakai.** Buka `src/App.java`, klik **Run** di atas `main`. Terminal harus menampilkan `Hello, World!`. Jika muncul error versi Java, jalankan **Java: Configure Java Runtime** dari Command Palette dan pilih JDK 21. Setelah berhasil, **hapus** `App.java`.

**Langkah 4 – Buat folder package.** Di panel Explorer, klik kanan `src` → **New Java Package**, lalu ketik nama berikut satu per satu (garis miring langsung membuat folder package bertingkat):

- `shop/model`
- `shop/shipping`
- `shop/discount`
- `shop/payment`
- `shop/service`
- `shop/app`

**Langkah 5 – Salin alat bantu.** Ekstrak zip ke folder lain, lalu salin isinya ke posisi yang **sama persis** di dalam `src` proyek Anda:

| Dari zip | Ke proyek Anda | Kegunaan |
|---|---|---|
| `src/selfcheck/` | `src/selfcheck/` | `SelfCheck` dan `TugasCheck` |
| `src/tools/` | `src/tools/` | `NimParams` untuk tugas |
| `src/review/` | `src/review/` | `BadShop` untuk tugas T3 |
| `src/shop/StudentInfo.java` | `src/shop/StudentInfo.java` | Identitas dan parameter tugas |

Struktur `src` yang benar setelah langkah ini:

```
src/
├── review/BadShop.java
├── selfcheck/SelfCheck.java
```

```
├── selfcheck/TugasCheck.java
├── shop/
│   ├── StudentInfo.java
│   ├── app/        discount/   model/
│   └── payment/    service/    shipping/
└── tools/NimParams.java
```

**Langkah 6 – Jalankan `SelfCheck`.** Buka `src/selfcheck/SelfCheck.java`, klik **Run**. Hasil yang benar saat ini: `0 / 23 pemeriksaan lulus`.

```
=== SELF CHECK: Praktikum Abstraction ===


-- L1 - Abstract class & abstract method
  [GAGAL] L1.1 Product adalah abstract class
          -> class/interface belum ada: shop.model.Product
  [GAGAL] L1.2 Product punya abstract method category()
          -> class/interface belum ada: shop.model.Product
...
=== HASIL: 0 / 23 pemeriksaan lulus ===
```

**Langkah 7 – Cara membuat class baru (dipakai sepanjang latihan).** Klik kanan folder package, misalnya `shop/model` → **New Java File** → **Class**, lalu ketik nama class, misalnya `Product`. Periksa baris pertama file: harus `package shop.model;`, sesuai foldernya. Untuk interface, pilih **Interface**. Untuk abstract class, pilih **Class** lalu tambahkan `abstract` sendiri.

**Kesalahan umum L0**

| Gejala | Penyebab | Perbaikan |
|---|---|---|
| `The declared package "shop.model" does not match the expected package` | Yang dibuka di VS Code adalah folder `src` atau folder induk, bukan folder proyek | **File → Open Folder** ke folder `abstraction-<NIM>` |
| `class Product is public, should be declared in a file named Product.java` | Nama file berbeda dengan nama class `public` | Samakan nama file dan nama class, termasuk huruf besar/kecil |
| `SelfCheck` tidak punya tombol **Run** | File disalin ke luar `src`, atau disalin bersama folder `src` sehingga menjadi `src/src/...` | Pindahkan sehingga strukturnya sama dengan gambar di Langkah 5 |
| Error `release version 21 not supported` | VS Code memakai JDK lama | **Java: Configure Java Runtime** → pilih JDK 21 |

Alternatif tanpa Extension Pack (terminal): buat folder proyek dan subfolder di atas secara manual, salin alat bantu, lalu kompilasi dengan perintah di bagian "Cara memakai SelfCheck".

## L1 – Abstract class dan abstract method (25 menit)

**Analogi.** Kasir toko tidak pernah menjual "produk" saja; yang dijual selalu kopi kemasan, rendang, atau e-book. "Produk" adalah konsep umum yang mengikat semua jenis barang, tetapi tidak pernah berwujud sendiri. Itulah abstract class, class yang **tidak bisa dibuat objeknya** dan menjadi cetakan bersama bagi subclass-nya. Abstract method adalah janji tanpa isi: "setiap produk pasti punya kategori", tetapi isinya ditentukan oleh tiap subclass.

**L1.a – `Product` (contoh lengkap, ketik ulang).** File `src/shop/model/Product.java`:

```
package shop.model;

public abstract class Product {
```

```
    private final String sku;
    private final String name;
    private final long price; // in rupiah

    protected Product(String sku, String name, long price) {
        if (sku == null || sku.isBlank()) {
            throw new IllegalArgumentException("SKU must not be blank");
        }
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name must not be blank");
        }
        if (price < 0) {
            throw new IllegalArgumentException("Price must not be negative");
        }
        this.sku = sku;
        this.name = name;
        this.price = price;
    }

    public String getSku() { return sku; }
    public String getName() { return name; }
    public long getPrice() { return price; }

    /** Price the customer actually pays. Subclasses may override it. */
    public long getFinalPrice() { return price; }

    /** Every concrete product must state its category. */
    public abstract String category();

    /** A concrete method that relies on the abstract method category(). */
    public String getInfo() {
        return "[" + category() + "] " + name + " - Rp" + getFinalPrice();
    }
}
```

Perhatikan tiga hal:

1. Abstract class tetap boleh punya constructor, field, dan method biasa;
2. Constructor-nya `protected` karena hanya dipanggil subclass lewat `super(...)`;
3. Method `getInfo()` memanggil method `category()` walaupun isinya belum ada. Pemanggilan itu sah karena setiap objek nyata pasti berasal dari subclass konkret.

**L1.b – Class `PhysicalProduct` (TULIS SENDIRI).** File `src/shop/model/PhysicalProduct.java`:

- `extends Product`, constructor `PhysicalProduct(String sku, String name, long price, int weightGrams)`.
- Tolak `weightGrams <= 0` dengan `IllegalArgumentException`.
- Method `category()` mengembalikan `"Fisik"`.

**L1.c – Class `DigitalProduct` (TULIS SENDIRI).** File `src/shop/model/DigitalProduct.java`:

- `extends Product`, konstruktor `DigitalProduct(String sku, String name, long price, String downloadUrl)`; `downloadUrl` tidak boleh kosong.
- Method `public String getDownloadUrl()`; `category()` mengembalikan `"Digital"`.

**L1.d – Class `ShippingService` (contoh lengkap).** File `src/shop/shipping/ShippingService.java`. Semua kurir memakai rumus yang sama; yang berbeda hanya tarif per kilogram.

```
package shop.shipping;

public abstract class ShippingService {
    private final String name;
```

```
    protected ShippingService(String name) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name must not be blank");
        }
        this.name = name;
    }

    public String getName() { return name; }

    /** Rate in rupiah per started kilogram. */
    protected abstract long ratePerKg();

    /** Weight is rounded up: 700 g -> 1 kg, 1001 g -> 2 kg. */
    public long calculateCost(int weightGrams) {
        if (weightGrams <= 0) {
            return 0;
        }
        long kilograms = (weightGrams + 999L) / 1000;
        return kilograms * ratePerKg();
    }
}
```

**L1.e – Class `RegularShipping` dan `ExpressShipping`.** Class `RegularShipping` disediakan; Class `ExpressShipping` **TULIS SENDIRI** dengan nama `"Ekspres"` dan tarif per Kg `18_000`.

```
package shop.shipping;

public class RegularShipping extends ShippingService {
    public RegularShipping() {
        super("Reguler");
    }

    @Override
    protected long ratePerKg() {
        return 10_000;
    }
}
```

**Uji pemahaman.** Tulis `Product p = new Product("X", "X", 1000);` di sebuah `main`, kompilasi, lalu catat pesan error-nya untuk laporan. Setelah itu hapus baris tersebut.

**Kesalahan umum L1**

| Salah | Benar | Alasan |
|---|---|---|
| `public abstract String category() { }` | `public abstract String category();` | Abstract method tidak punya badan, diakhiri titik koma |
| Lupa menulis `abstract` pada class yang punya abstract method | `public abstract class Product` | Compiler menolak class konkret yang punya method tanpa isi |
| `super(...)` ditulis setelah baris lain di konstruktor subclass | `super(...)` di baris pertama | Bagian induk harus dibangun lebih dulu |
| Field `protected` agar mudah diakses subclass | Field `private` + getter | Subclass tidak boleh melewati validasi induk (enkapsulasi) |

**Checkpoint L1:** SelfCheck L1.1–L1.7 lulus.

```
-- L1 - Abstract class & abstract method
  [LULUS] L1.1 Product adalah abstract class
  [LULUS] L1.2 Product punya abstract method category()
  [LULUS] L1.3 PhysicalProduct & DigitalProduct adalah subclass konkret Product
  [LULUS] L1.4 Konstruktor menolak harga negatif (IllegalArgumentException)
  [LULUS] L1.5 getInfo() PhysicalProduct sesuai format
  [LULUS] L1.6 ShippingService abstract dengan abstract method ratePerKg()
```

```
  [LULUS] L1.7 calculateCost(): Reguler 700g=10000, 1001g=20000, 0g=0; Ekspres 700g=18000

=== HASIL: 7 / 23 pemeriksaan lulus ===
```

## L2 – Interface dan program to an interface (25 menit)

**Analogi.** Kasir tidak perlu tahu cara kerja server DompetKu atau sistem bank. Ia hanya butuh satu tombol: "bayar sejumlah sekian". Interface (antarmuka) adalah daftar kemampuan seperti tombol itu: kontrak berisi method yang **wajib** disediakan oleh setiap class yang menandatanganinya dengan `implements`. Bedanya dengan abstract class: abstract class menjawab "*ini termasuk jenis apa?*" (is-a), sedangkan interface menjawab "*ini bisa melakukan apa?*" (can-do). `PhysicalProduct` adalah `Product` dan sekaligus *bisa dikirim* (`Shippable`).

**L2.a – `Shippable` (contoh).** File `src/shop/shipping/Shippable.java`:

```
package shop.shipping;

/** Capability: anything that has weight and can be shipped. */
public interface Shippable {
    int getWeightGrams();
}
```

Ubah `PhysicalProduct` menjadi `public class PhysicalProduct extends Product implements Shippable`, lalu beri `@Override` pada `getWeightGrams()`. `DigitalProduct` **tidak** mengimplementasikan `Shippable` produk digital karena tidak dikirim.

**L2.b – Interface `PaymentMethod` (contoh).** File `src/shop/payment/PaymentMethod.java`:

```
package shop.payment;

/** Contract for every payment method. */
public interface PaymentMethod {
    String getName();

    /** @return true if the payment succeeds */
    boolean pay(long amount);
}
```

**L2.c – Class `EWalletPayment` (TULIS SENDIRI).** Package `shop.payment`, `implements PaymentMethod`:

- Constructor `EWalletPayment(String provider, long balance)`; tolak provider kosong atau saldo negatif.
- Method `getName()` mengembalikan `"E-Wallet " + provider`; tambahkan method `public long getBalance()`.
- Method `pay(amount)`: `return false` (gagal) jika `amount <= 0` atau melebihi saldo; jika berhasil, saldo dikurangi lalu `return true`.

**L2.d – Class `BankTransferPayment` (TULIS SENDIRI).** Constructor `BankTransferPayment(String bankName)`; Method `getName()` mengembalikan `"Transfer " + bankName`; Method `pay(amount)` mengembalikan `amount > 0` (simulasi).

**L2.e – Class `CheckoutService`.** File `src/shop/service/CheckoutService.java`. Dua method diberikan; dua method lainnya **TULIS SENDIRI**.

```
package shop.service;

import java.util.List;
import shop.model.Product;
```

```
import shop.payment.PaymentMethod;
import shop.shipping.Shippable;
import shop.shipping.ShippingService;

public class CheckoutService {

    public long subtotal(List<Product> items) {
        long total = 0;
        for (Product product : items) {
            total += product.getFinalPrice();
        }
        return total;
    }

    public int totalWeight(List<Product> items) {
        int grams = 0;
        for (Product product : items) {
            if (product instanceof Shippable shippable) { // a capability, not a class
                grams += shippable.getWeightGrams();
            }
        }
        return grams;
    }

    // TODO total(List<Product> items, ShippingService shipping)
    // TODO checkout(List<Product> items, ShippingService shipping, PaymentMethod payment)
}
```

- `long total(List<Product> items, ShippingService shipping)` = `subtotal(items)` + ongkir dari `totalWeight(items)`.
- `boolean checkout(List<Product> items, ShippingService shipping, PaymentMethod payment)`: kembalikan `false` jika keranjang kosong; selain itu bayar `total(...)` dengan `payment.pay(...)`.

Inilah program to an interface (memprogram terhadap antarmuka): `CheckoutService` hanya mengenal `PaymentMethod` dan `ShippingService`. Metode pembayaran baru bisa ditambahkan tanpa mengubah satu baris pun di `CheckoutService`.

**L2.f – ShopApp (mulai ditulis).** Buat `src/shop/app/ShopApp.java` dengan `main` berisi keranjang contoh berikut, lalu cetak `getInfo()` tiap produk, subtotal, berat, dan total kedua kurir.

```
package shop.app;

import java.util.List;
import shop.model.DigitalProduct;
import shop.model.PhysicalProduct;
import shop.model.Product;
import shop.service.CheckoutService;
import shop.shipping.ExpressShipping;
import shop.shipping.RegularShipping;
import shop.shipping.ShippingService;

public class ShopApp {
    public static void main(String[] args) {
        // L1: abstract class and abstract method
        Product coffee = new PhysicalProduct("KOPI-001", "Kopi Arabika Gayo 250g", 85_000,
300);
        Product rendang = new PhysicalProduct("RDG-002", "Rendang Kemasan 250g", 120_000,
400);
        Product ebook = new DigitalProduct("EBK-003", "E-book Resep Kue Nusantara", 50_000,
                "https://toko-umkm.example/unduh/ebk-003");

        List<Product> cart = List.of(coffee, rendang, ebook);
```

```
        System.out.println("== Katalog ==");
        for (Product product : cart) {
            System.out.println(product.getInfo());
        }

        // L2: interface and program to an interface
        CheckoutService checkout = new CheckoutService();
        ShippingService regular = new RegularShipping();
        ShippingService express = new ExpressShipping();
        System.out.println("\n== Ringkasan ==");
        System.out.println("Subtotal      : Rp" + checkout.subtotal(cart));
        System.out.println("Berat kiriman : " + checkout.totalWeight(cart) + " g");
        System.out.println("Total Reguler : Rp" + checkout.total(cart, regular));
        System.out.println("Total Ekspres : Rp" + checkout.total(cart, express));
    }
}
```

Contoh output `ShopApp` setelah L2:

```
== Katalog ==
[Fisik] Kopi Arabika Gayo 250g - Rp85000
[Fisik] Rendang Kemasan 250g - Rp120000
[Digital] E-book Resep Kue Nusantara - Rp50000

== Ringkasan ==
Subtotal      : Rp255000
Berat kiriman : 700 g
Total Reguler : Rp265000
Total Ekspres : Rp273000
```

Pada titik ini total Reguler masih **265000** dan e-book masih Rp50000, karena diskon e-book baru muncul di L3. Setelah L3 selesai, nilainya menjadi **260000**. Cara menghitung: subtotal 255000 + ongkir Reguler 10000 (700 g dibulatkan menjadi 1 kg × Rp10000).

**Kesalahan umum L2**

| Salah | Benar | Alasan |
|---|---|---|
| `checkout(..., EWalletPayment payment)` | `checkout(..., PaymentMethod payment)` | Parameter class konkret menutup pintu bagi metode pembayaran lain |
| `if (product instanceof PhysicalProduct p)` di `CheckoutService` | `if (product instanceof Shippable s)` | Bergantung pada kemampuan, bukan pada satu class tertentu |
| `int getWeightGrams();` ditulis tanpa `public` di class implementasi | `public int getWeightGrams()` | Method interface otomatis `public`; implementasinya tidak boleh lebih tertutup |
| `new PaymentMethod()` | `new EWalletPayment(...)` | Interface tidak bisa dibuat objeknya (kecuali sebagai anonymous class, lihat L5) |

**Checkpoint L2:** SelfCheck L2.1–L2.4 lulus. L2.5 baru lulus setelah L3.

```
-- L2 - Interface & program to an interface
  [LULUS] L2.1 Shippable interface; PhysicalProduct implements, DigitalProduct tidak
  [LULUS] L2.2 PaymentMethod interface dengan getName() dan pay(long)
  [LULUS] L2.3 EWalletPayment: bayar sukses mengurangi saldo, saldo kurang ditolak
  [LULUS] L2.4 CheckoutService.checkout(...) menerima PaymentMethod & ShippingService (bukan
class konkret)
  [GAGAL] L2.5 Total keranjang contoh: Reguler=260000, Ekspres=268000
          -> hasil tidak sesuai spesifikasi

=== HASIL: 11 / 23 pemeriksaan lulus ===
```

## L3 – Interface lanjutan: default, static, private method dan interface inheritance (25 menit)

**Analogi.** Program promo toko punya aturan umum: "harga dikurangi sekian persen, persentase tidak boleh negatif atau di atas 100". Setiap produk promo cukup menyebut persentasenya, sedangkan cara menghitung potongan sudah disediakan. Di Java 8+, interface boleh membawa perilaku bawaan seperti itu:

| Jenis method di interface | Sejak | Fungsi | Harus di-override? |
|---|---|---|---|
| abstract (biasa) | Java 1.0 | Kontrak yang wajib diisi | Ya |
| `default` | Java 8 | Perilaku bawaan yang diwarisi gratis | Tidak, boleh |
| `static` | Java 8 | Utilitas milik interface, dipanggil `NamaInterface.method()` | Tidak bisa |
| `private` | Java 9 | Pembantu internal bagi default method lain | Tidak terlihat dari luar |

**L3.a – Class `Discountable` (contoh lengkap).** File `src/shop/discount/Discountable.java`:

```
package shop.discount;

public interface Discountable {

    int discountPercent(); // abstract: each class decides

    default long applyDiscount(long price) {
        return price - percentOf(price, safePercent());
    }

    static long percentOf(long amount, int percent) {
        return amount * percent / 100;
    }

    private int safePercent() {
        int percent = discountPercent();
        if (percent < 0) {
            return 0;
        }
        return Math.min(percent, 100);
    }
}
```

**L3.b – Class `DigitalProduct` ikut promo (TULIS SENDIRI).** Ubah menjadi `extends Product implements Discountable`:

- `discountPercent()` mengembalikan konstanta `private static final int DIGITAL_DISCOUNT_PERCENT = 10`.
- Override `getFinalPrice()` agar mengembalikan `applyDiscount(getPrice())`. Harga e-book 50000 menjadi 45000.

Perhatikan bahwa `CheckoutService` tidak perlu diubah: `subtotal()` sudah memanggil `getFinalPrice()` (polimorfisme).

**L3.c – Interface inheritance.** Sebuah interface dapat mewarisi interface lain dengan kata kunci **`extends`** (bukan `implements`). Buat dua interface (`RefundablePayment` dan `RefundablePayment`) di package `shop.payment`:

```
package shop.payment;

public interface RefundablePayment extends PaymentMethod {
    boolean refund(long amount);
}
```

```
package shop.payment;

public interface InstallmentPayment extends PaymentMethod {
    int maxInstallmentMonths();

    /** Monthly amount, rounded up. */
    default long monthlyInstallment(long amount, int months) {
        if (months < 1 || months > maxInstallmentMonths()) {
            throw new IllegalArgumentException(
                    "Months must be between 1 and " + maxInstallmentMonths());
        }
        return (amount + months - 1) / months;
    }
}
```

**L3.d – (TULIS SENDIRI)**

1. Ubah `EWalletPayment` menjadi `implements RefundablePayment`. Method `refund(amount)`: akan mengembalikan nilai false (`return false`) jika `amount <= 0`; dan `return true` jika saldo bertambah.
2. Buat Class `CreditCardPayment implements RefundablePayment, InstallmentPayment` (satu class, **dua** interface) yang terdiri dari:
    - Constructor `CreditCardPayment(String cardHolder, long creditLimit)`; Method `getName()` mengembalikan `"Kartu Kredit (" + cardHolder + ")"`.
    - Field `usedCredit`; `pay` gagal jika `amount <= 0` atau `usedCredit + amount > creditLimit`.
    - `refund` gagal jika `amount <= 0` atau melebihi `usedCredit`; jika berhasil `usedCredit` berkurang.
    - `maxInstallmentMonths()` = 12; tambahkan `getUsedCredit()`.
3. Tambahkan ke `CheckoutService` method `public boolean refund(RefundablePayment payment, long amount)` yang meneruskan ke `payment.refund(amount)`.
4. Di `ShopApp`, coba `checkout.refund(new BankTransferPayment("Bank Nagari"), 1_000);`. Catat error compiler-nya, lalu jadikan komentar.

Poin keempat menunjukkan kekuatan tipe: compiler sudah menolak refund via transfer bank **sebelum** program berjalan, tanpa satu pun `if`.

**L3.e – Lengkapi `ShopApp` dengan pembayaran.** Tambahkan kode berikut di akhir `main`, lalu tambahkan `import` untuk `EWalletPayment`, `BankTransferPayment`, `CreditCardPayment`, dan `PaymentMethod`.

```
EWalletPayment wallet = new EWalletPayment("DompetKu", 300_000);
PaymentMethod transfer = new BankTransferPayment("Bank Nagari");
System.out.println("\n== Pembayaran ==");
System.out.println(wallet.getName() + " berhasil? " + checkout.checkout(cart, regular,
wallet));
System.out.println("Sisa saldo: Rp" + wallet.getBalance());
System.out.println(transfer.getName() + " berhasil? " + checkout.checkout(cart, express,
transfer));

// L3: interface inheritance, multiple interfaces, default method
CreditCardPayment card = new CreditCardPayment("Siti Rahma", 1_000_000);
System.out.println(card.getName() + " berhasil? " + checkout.checkout(cart, express, card));
System.out.println("Cicilan 3 bulan: Rp" + card.monthlyInstallment(checkout.total(cart,
express), 3));

System.out.println("\n== Refund e-book ==");
System.out.println("Refund e-wallet berhasil? " + checkout.refund(wallet,
ebook.getFinalPrice()));
```

```
System.out.println("Saldo setelah refund: Rp" + wallet.getBalance());
// checkout.refund(transfer, 1_000); // compile error: BankTransferPayment is not
RefundablePayment
```

Contoh output `ShopApp` setelah L3 (perhatikan harga e-book kini Rp45000):

```
== Katalog ==
[Fisik] Kopi Arabika Gayo 250g - Rp85000
[Fisik] Rendang Kemasan 250g - Rp120000
[Digital] E-book Resep Kue Nusantara - Rp45000

== Ringkasan ==
Subtotal      : Rp250000
Berat kiriman : 700 g
Total Reguler : Rp260000
Total Ekspres : Rp268000

== Pembayaran ==
E-Wallet DompetKu berhasil? true
Sisa saldo: Rp40000
Transfer Bank Nagari berhasil? true
Kartu Kredit (Siti Rahma) berhasil? true
Cicilan 3 bulan: Rp89334

== Refund e-book ==
Refund e-wallet berhasil? true
Saldo setelah refund: Rp85000
```

Cara membaca angkanya: saldo DompetKu 300000 − total Reguler 260000 = sisa **40000**. Cicilan 3 bulan = 268000 ÷ 3 dibulatkan ke atas = **89334**. Refund e-book 45000 membuat saldo menjadi **85000**.

**Kesalahan umum L3**

| Salah | Benar | Alasan |
|---|---|---|
| `interface RefundablePayment implements PaymentMethod` | `interface RefundablePayment extends PaymentMethod` | Interface mewarisi interface dengan `extends` |
| `class CreditCardPayment extends RefundablePayment, InstallmentPayment` | `implements RefundablePayment, InstallmentPayment` | Class hanya bisa `extends` satu class, tetapi bisa `implements` banyak interface |
| Memanggil `percentOf(...)` lewat objek: `ebook.percentOf(...)` | `Discountable.percentOf(...)` | Static method interface tidak diwarisi oleh class |
| Menaruh field yang berubah di interface | Field di class implementasi | Field interface otomatis `public static final` (konstanta) |

**Checkpoint L3:** SelfCheck L2.5 dan L3.1–L3.6 lulus.

```
-- L2 - Interface & program to an interface
  [LULUS] L2.5 Total keranjang contoh: Reguler=260000, Ekspres=268000

-- L3 - Default, static, private method; interface inheritance
  [LULUS] L3.1 Discountable: applyDiscount default, percentOf static, ada private method
  [LULUS] L3.2 DigitalProduct implements Discountable; harga akhir 50000 -> 45000
  [LULUS] L3.3 RefundablePayment & InstallmentPayment adalah interface turunan PaymentMethod
  [LULUS] L3.4 CreditCardPayment implements RefundablePayment DAN InstallmentPayment
  [LULUS] L3.5 monthlyInstallment(100000, 3)=33334 dan 13 bulan ditolak
  [LULUS] L3.6 CheckoutService.refund(...) hanya menerima RefundablePayment

=== HASIL: 18 / 23 pemeriksaan lulus ===
```

## L4 – `final` method dan `final` class (10 menit)

**Analogi.** Rumus ongkir toko ditetapkan pemilik toko: berat dibulatkan ke atas per kilogram, lalu dikali tarif. Kurir boleh menentukan tarifnya sendiri, tetapi **tidak boleh** mengganti rumusnya, misalnya tidak membulatkan berat. Kata kunci `final` adalah kunci gembok untuk keputusan desain seperti itu:

- `final` method: tidak bisa di-override oleh subclass.
- `final` class: tidak bisa diwarisi sama sekali.

**L4.a – Kunci rumus ongkir.** Ubah method di `ShippingService` menjadi:

```
public final long calculateCost(int weightGrams) {
```

Uji: buat untuk sementara class `FreeShipping extends ShippingService` yang meng-override `calculateCost`. Catat error compiler-nya, lalu hapus class tersebut. Subclass tetap bebas mengatur `ratePerKg()`.

**L4.b – `Address` immutable (TULIS SENDIRI).** File `src/shop/model/Address.java`. Immutable (tak dapat diubah) berarti nilai objek tidak pernah berubah setelah dibuat.

- `public final class Address` dengan field `private final String street, city, postalCode`.
- Constructor `Address(String street, String city, String postalCode)` menolak nilai kosong.
- Hanya getter, **tanpa setter**. Method `toString()` menghasilkan `street + ", " + city + " " + postalCode`.

Mengapa class-nya juga `final`? Tanpa `final`, orang lain dapat membuat `class EditableAddress extends Address` yang menambah setter. Objek turunan itu tetap diterima di mana pun `Address` diminta, sehingga jaminan immutable bocor.

**Kesalahan umum L4**

| Salah | Benar | Alasan |
|---|---|---|
| `final` pada `ratePerKg()` | `final` pada `calculateCost()` saja | Yang dikunci adalah rumus bersama, bukan bagian yang memang harus berbeda |
| `abstract final class ...` | Pilih salah satu | `abstract` menuntut diwarisi, `final` melarangnya |
| `final class` dengan field `private` biasa + setter | Semua field `private final`, tanpa setter | `final` pada class tidak membuat field-nya tak berubah |
| Mengira `final` pada variabel referensi membuat objeknya immutable | `final` hanya mengunci referensi | `final List<...>` tetap bisa diisi `add(...)` |

**Checkpoint L4:** SelfCheck L4.1–L4.2 lulus. Output `ShopApp` sama persis dengan L3, karena `final` tidak mengubah perilaku program; yang berubah adalah apa yang **diizinkan** compiler. Contoh output SelfCheck:

```
-- L4 - final method & final class
  [LULUS] L4.1 ShippingService.calculateCost(int) adalah final method
  [LULUS] L4.2 Address final class, semua field private final, tanpa setter

=== HASIL: 20 / 23 pemeriksaan lulus ===
```

## L5 – Nested class: static nested, inner, dan anonymous class (20 menit)

**Analogi.** Baris pesanan ("Kopi Gayo x2") hanya bermakna sebagai bagian dari pesanan, tetapi tidak perlu tahu pesanan mana yang memilikinya. Struk berbeda: struk selalu milik **satu** pesanan tertentu dan harus

membaca nomor pesanan serta alamatnya. Voucher promo hari ini hanya dipakai sekali dan tidak perlu diberi nama class.

| Jenis | Bentuk | Butuh objek luar? | Kapan dipakai |
|---|---|---|---|
| Static nested class | `static class Line` di dalam `Order` | Tidak | Class pembantu yang hanya relevan bagi class luarnya |
| Inner class (non-static) | `class Receipt` di dalam `Order` | Ya, menyimpan referensi ke objek luar | Perlu membaca data objek luar (`Order.this`) |
| Anonymous class | `new PaymentMethod() { ... }` | Tergantung tempat dibuat | Implementasi sekali pakai, tanpa nama |

**L5.a – Class `Order` (sebagian diberikan).** File `src/shop/model/Order.java`:

```
package shop.model;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Order {
    private final String orderId;
    private final Address shippingAddress;
    private final List<Line> lines = new ArrayList<>();

    public Order(String orderId, Address shippingAddress) {
        if (orderId == null || orderId.isBlank() || shippingAddress == null) {
            throw new IllegalArgumentException("Order id and address are required");
        }
        this.orderId = orderId;
        this.shippingAddress = shippingAddress;
    }

    public String getOrderId() { return orderId; }
    public Address getShippingAddress() { return shippingAddress; }

    public void addLine(Product product, int quantity) {
        lines.add(new Line(product, quantity));
    }

    public List<Line> getLines() { return Collections.unmodifiableList(lines); }

    // TODO getItemsTotal(), createReceipt(), class Line, class Receipt
}
```

**L5.b – (TULIS SENDIRI) di dalam Class `Order`**

1. `public static class Line`: field `private final Product product` dan `private final int quantity`; constructor menolak `product == null` atau `quantity <= 0`; getter; `long subtotal()` = `product.getFinalPrice() * quantity`.
2. `public long getItemsTotal()`: jumlah `subtotal()` semua baris.
3. `public class Receipt` (tanpa `static`) dengan method `public String toText()` yang memuat: `STRUK PESANAN <orderId>`, satu baris per item (`- <nama> x<qty> = Rp<subtotal>`), `Total pesanan: Rp<total>`, dan `Kirim ke: <alamat>`. Baca field luar dengan `Order.this.orderId` dan `Order.this.shippingAddress`.
4. `public Receipt createReceipt()` yang mengembalikan `new Receipt()`.

**L5.c – Lengkapi `ShopApp`.** Tambahkan alamat, pesanan, struk, dan voucher:

```
Address address = new Address("Jl. Moh Hatta No. 10", "Padang", "25121");
Order order = new Order("ORD-0001", address);
order.addLine(coffee, 2);
order.addLine(ebook, 1);
Order.Receipt receipt = order.new Receipt(); // inner object needs an outer object
System.out.println(receipt.toText());

PaymentMethod voucher = new PaymentMethod() { // anonymous class
    private long remaining = 50_000;

    @Override
    public String getName() { return "Voucher Promo Hari Ini"; }

    @Override
    public boolean pay(long amount) {
        if (amount <= 0 || amount > remaining) {
            return false;
        }
        remaining -= amount;
        return true;
    }
};
System.out.println("\n" + voucher.getName() + " untuk e-book berhasil? "
        + checkout.checkout(List.of(ebook), regular, voucher));
```

Perhatikan sintaks `order.new Receipt()`: objek inner class hanya bisa lahir dari objek luar. Sebaliknya, `new Order.Line(coffee, 1)` dapat dibuat tanpa objek `Order` karena `Line` bersifat static.
Contoh output lengkap `ShopApp` setelah L5. Bagian struk dan voucher adalah tambahan L5; bagian lain sama dengan L3. Voucher berhasil karena total e-book (45000, tanpa ongkir karena tidak dikirim) masih di bawah nilai voucher 50000:

```
== Katalog ==
[Fisik] Kopi Arabika Gayo 250g - Rp85000
[Fisik] Rendang Kemasan 250g - Rp120000
[Digital] E-book Resep Kue Nusantara - Rp45000

== Ringkasan ==
Subtotal      : Rp250000
Berat kiriman : 700 g
Total Reguler : Rp260000
Total Ekspres : Rp268000

== Pembayaran ==
E-Wallet DompetKu berhasil? true
Sisa saldo: Rp40000
Transfer Bank Nagari berhasil? true
Kartu Kredit (Siti Rahma) berhasil? true
Cicilan 3 bulan: Rp89334

== Refund e-book ==
Refund e-wallet berhasil? true
Saldo setelah refund: Rp85000

STRUK PESANAN ORD-0001
- Kopi Arabika Gayo 250g x2 = Rp170000
- E-book Resep Kue Nusantara x1 = Rp45000
Total barang: Rp215000
Kirim ke: Jl. Moh Hatta. 10, Padang 25121

Voucher Promo Hari Ini untuk e-book berhasil? true
```

**Kesalahan umum L5**

| Salah | Benar | Alasan |
|---|---|---|
| `Line` dibuat inner class (tanpa `static`) | `static class Line` | Line tidak memakai data `Order`; inner class menyimpan referensi tersembunyi ke objek luar tanpa guna |
| `new Order.Receipt()` | `order.new Receipt()` atau `order.createReceipt()` | Inner class butuh objek luar |
| Di `Receipt` menulis `this.orderId` | `Order.this.orderId` (atau `orderId` saja) | Di dalam inner class, `this` adalah objek `Receipt`, bukan `Order` |
| Anonymous class dipakai untuk logika yang dipakai berkali-kali | Class bernama | Anonymous class sulit diuji dan tidak bisa dipakai ulang |

**Checkpoint L5:** SelfCheck L5.1–L5.3 lulus. Target akhir: **23 / 23**. Contoh output:

```
-- L5 - Static nested, inner, dan anonymous class
  [LULUS] L5.1 Order.Line static nested; Order.Receipt inner (non-static)
  [LULUS] L5.2 Receipt.toText() memuat id pesanan, total barang, dan kota tujuan
  [LULUS] L5.3 ShopApp memakai anonymous class yang mengimplementasikan PaymentMethod

=== HASIL: 23 / 23 pemeriksaan lulus ===
```

# D. Pengumpulan Latihan (akhir sesi)

Kumpulkan sebelum sesi ditutup, walaupun belum semua pemeriksaan lulus. Pekerjaan yang belum selesai tetap dinilai sesuai bagian yang sudah benar.

1. Jalankan `SelfCheck` dan ambil tangkapan layar (*screenshot*) hasil akhirnya (baris `=== HASIL: x / 23 ...`).
2. Commit & push semua kode program Latihan Praktikum 6 ke repository masing-masing, tempatkan di: **<YOUR_REPOSITORY>/Praktikum6/Latihan**
3. Zip folder `src` dengan nama `LATIHAN_<NIM>_<Nama>.zip`.
4. Unggah zip dan tangkapan layar (*screenshot*) ke aplikasi CODATA LAB Pengumpulan Latihan Praktikum 6.

Pesan *error compiler* yang diminta pada L1 (membuat objek `Product`), L3 (refund lewat transfer bank), dan L4 (override `calculateCost`) dilampirkan pada **laporan tugas**, bukan pada pengumpulan latihan.

# E. Tugas Rumah (individu)

Tugas melanjutkan proyek latihan Anda. Fokusnya bergeser dari *menulis* konstruksi ke *merancang* dan *menilai* desain. **Deadline**: **6x24 jam setelah pelaksanaan praktikum.** Semua contoh output di bagian ini memakai NIM contoh **2401010057**. Angka biaya admin dan panjang SMS Anda akan berbeda sesuai NIM masing-masing, namun struktur output-nya tetap sama.

### E.0 Parameter pribadi berbasis NIM

Setiap mahasiswa memakai angka yang berbeda, sehingga hasil program juga berbeda.

1. Buka **Terminal → New Terminal** di VS Code, lalu jalankan `java -cp bin tools.NimParams <NIM Anda>`. Ketik NIM **tanpa** tanda `< >`, misalnya `java -cp bin tools.NimParams 2401010057`. Tombol **Run** tidak bisa memberi argumen NIM. Jika muncul `Could not find or load main class`, lihat tabel penyebabnya di Bagian A.
2. Salin ketiga nilai yang dicetak ke `src/shop/StudentInfo.java`, lalu isi juga `NIM` dan `NAME`.

Contoh output untuk NIM 2401010057:

```
Parameter tugas untuk NIM 2401010057
  CARD_ADMIN_PERCENT  = 2
  TRANSFER_ADMIN_FEE  = 3500
  SMS_MAX_LENGTH      = 157
```

| Konstanta | Rumus (d = digit terakhir NIM, dd = dua digit terakhir) | Contoh NIM 2401010057 |
|---|---|---|
| CARD_ADMIN_PERCENT | 1 + (d mod 3) | 2 |
| TRANSFER_ADMIN_FEE | 2500 + 500 × (d mod 5) | 3500 |
| SMS_MAX_LENGTH | 100 + dd | 157 |

`TugasCheck` memeriksa apakah konstanta Anda cocok dengan NIM yang tertulis. Konstanta yang tidak cocok dengan NIM penulisnya dianggap indikasi penyalinan.

### T1 – Evolusi API dan konflik diamond (bobot 25)

**Skenario.** Toko sudah berjalan dengan tiga metode pembayaran. Tim keuangan kemudian meminta fitur baru yaitu biaya admin per metode pembayaran. Di industri, ini disebut API evolution (evolusi antarmuka): kontrak yang sudah dipakai banyak class harus berubah tanpa merusak semuanya.

**T1.a – Cara yang salah dulu.** Tambahkan method abstract `long adminFee(long amount);` ke `PaymentMethod`, lalu kompilasi. *Screenshot* semua error dan jawab di laporan: ada berapa error, class mana saja, dan mengapa anonymous class voucher di `ShopApp` juga ikut rusak?

**T1.b – Default method.** Ubah method tersebut menjadi default:

```
default long adminFee(long amount) {
    return 0;
}
```

Kompilasi ulang: semua error hilang tanpa menyentuh class implementasi.

**T1.c – Override sesuai kebutuhan.**

- `CreditCardPayment.adminFee(amount)` = `amount * StudentInfo.CARD_ADMIN_PERCENT / 100`.
- `BankTransferPayment.adminFee(amount)` = `StudentInfo.TRANSFER_ADMIN_FEE` (biaya tetap).
- `EWalletPayment` tidak diubah; biaya adminnya 0.

**T1.d – `CheckoutService`.** Tambahkan `public long grandTotal(List<Product> items, ShippingService shipping, PaymentMethod payment)` = `total(items, shipping)` + `payment.adminFee(total)`. Ubah `checkout(...)` agar membayar `grandTotal(...)`.

**T1.e – Konflik diamond.** Tambahkan default method yang sama ke dua interface:

```
// in RefundablePayment
default String policy() {
    return "Refund maksimal 7 hari";
}

// in InstallmentPayment
default String policy() {
    return "Cicilan maksimal " + maxInstallmentMonths() + " bulan";
}
```

Kompilasi, lalu catat error pada `CreditCardPayment`. Selesaikan dengan meng-override `policy()` di `CreditCardPayment` sehingga hasilnya menggabungkan keduanya, dipisah `"; "`. Gunakan sintaks `RefundablePayment.super.policy()` dan `InstallmentPayment.super.policy()`.

Jawab di laporan:

1) Mengapa `EWalletPayment` tidak perlu meng-override `policy()`?
2) Mengapa Java tidak memilih salah satu default secara otomatis?

**Checkpoint T1:** `TugasCheck` P.1 dan T1.1–T1.4 lulus. Contoh output bagian awal `TugasCheck` untuk NIM 2401010057 (angka di tanda kurung mengikuti NIM Anda):

```
=== TUGAS CHECK: Praktikum Abstraction ===
NIM: 2401010057 | Contoh Mahasiswa

-- Parameter NIM
  [LULUS] P.1 Konstanta di StudentInfo sesuai NIM (2%, Rp3500, 157 karakter)

-- T1 - Evolusi API & diamond
  [LULUS] T1.1 PaymentMethod.adminFee(long) adalah default method; e-wallet = 0
  [LULUS] T1.2 Biaya admin kartu kredit & transfer sesuai parameter NIM
  [LULUS] T1.3 grandTotal(...) = total + biaya admin (kartu kredit, keranjang contoh,
Ekspres)
  [LULUS] T1.4 Diamond policy(): kartu kredit menggabungkan kedua kebijakan
```

Rumus cepat untuk mengecek angka Anda sendiri (kurir Ekspres, keranjang contoh): total sebelum biaya admin = 268000; grand total kartu kredit = 268000 + 268000 × `CARD_ADMIN_PERCENT` ÷ 100; grand total transfer = 268000 + `TRANSFER_ADMIN_FEE`; grand total e-wallet = 268000. Output `ShopApp` dari latihan tidak berubah setelah T1, karena semua saldo dan limit di sana masih cukup untuk membayar biaya admin.

### T2 – Merancang sistem notifikasi pesanan (bobot 40)

**Skenario.** Saat status pesanan berubah, pelanggan diberi tahu lewat email, SMS, dan WhatsApp. Tahun depan mungkin ditambah kanal lain (Telegram, push notification). Rancang sistem yang membuat kanal baru cukup ditambahkan sebagai class baru, tanpa mengubah kode pengirim.

**Langkah 1 – Rancang dulu (UML).** Buat class diagram UML untuk package `shop.notification` (boleh draw.io, PlantUML, atau gambar tangan yang rapi). Tunjukkan interface, abstract class, ketiga kanal, `NotificationService`, `CustomerContact`, serta relasi `implements`/`extends`/asosiasi. **Lampirkan di laporan.**

**Langkah 2 – Implementasikan** di package `shop.notification`. Class, variabel, dan method berikut **wajib** (dipakai `TugasCheck`), isi lainnya adalah keputusan desain Anda.

| Tipe | Spesifikasi minimum |
|---|---|
| `final class CustomerContact` | Constructor `(String name, String email, String phone)`; `name` wajib; `email`/`phone` boleh kosong (`null` diubah menjadi `""`); hanya getter |
| `interface NotificationChannel` | `String channelName()`; `boolean send(CustomerContact contact, String message)` |
| `abstract class AbstractChannel implements NotificationChannel` | `send(...)` bersifat **`final`**: menolak pesan kosong, mencari alamat tujuan, menyiapkan pesan, mengirim, lalu mencatat. Sediakan `int getSentCount()` dan `String getLastDelivered()` (pesan terakhir yang berhasil dikirim, setelah disiapkan) |
| `EmailChannel extends AbstractChannel` | Constructor tanpa parameter; tujuan = email kontak, hanya jika memuat `@` |
| `SmsChannel extends AbstractChannel` | Constructor `SmsChannel(int maxLength)`; tujuan = telepon yang diawali `08`; pesan lebih panjang dari `maxLength` dipotong menjadi **tepat** `maxLength` karakter dan diakhiri `"..."` |
| `WhatsAppChannel extends AbstractChannel` | Constructor tanpa parameter; tujuan = telepon berawalan `08` atau `+62` dengan panjang minimal 10; pesan diberi awalan `"[Toko UMKM] "` |

| Tipe | Spesifikasi minimum |
|---|---|
| `NotificationService` | `void register(NotificationChannel channel);` `int broadcast(CustomerContact contact, String message)` mengembalikan jumlah kanal yang berhasil |

Petunjuk desain: `send(...)` yang `final` memanggil method `protected abstract` yang diisi tiap kanal, misalnya `resolveRecipient(...)` dan `deliver(...)`, serta method `protected` biasa `prepare(...)` yang boleh di-override. Pola ini adalah cikal bakal Template Method pattern.

**Langkah 3 – Demo.** Buat `src/shop/app/TugasApp.java` yang:

1. menampilkan grand total keranjang contoh (kurir Ekspres) untuk tiga metode pembayaran;
2. mencetak `policy()` kartu kredit dan e-wallet;
3. mendaftarkan ketiga kanal (SMS memakai `StudentInfo.SMS_MAX_LENGTH`) **dan** satu kanal log konsol berupa anonymous class `NotificationChannel`;
4. mengirim satu pesan panjang (lebih dari 200 karakter) ke dua kontak: satu punya email dan nomor `08...`, satu hanya punya nomor `+62...`.

Contoh output `TugasApp` untuk NIM 2401010057. Nama, NIM, dan angka biaya admin mengikuti data Anda; isi pesan boleh berbeda asalkan lebih dari 200 karakter. Baris SMS dipotong tepat sepanjang `SMS_MAX_LENGTH` (di sini 157 karakter, termasuk `...`). Budi tidak menerima email dan SMS karena tidak punya email dan nomornya berawalan `+62`.

```
Tugas oleh Contoh Mahasiswa (2401010057)

== T1: Total dengan biaya admin (Ekspres) ==
E-Wallet DompetKu : Rp268000 (admin Rp0)
Transfer Bank Nagari : Rp271500 (admin Rp3500)
Kartu Kredit (Siti Rahma) : Rp273360 (admin Rp5360)

Kebijakan kartu kredit : Refund maksimal 7 hari; Cicilan maksimal 12 bulan
Kebijakan e-wallet     : Refund maksimal 7 hari

== T2: Notifikasi status pesanan ==
[EMAIL to siti@example.com] Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres. Nomor resi
akan muncul di aplikasi dalam 1x24 jam. Terima kasih telah berbelanja produk UMKM Sumatera
Barat di toko kami!
[SMS to 081234567890] Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres. Nomor resi akan
muncul di aplikasi dalam 1x24 jam. Terima kasih telah berbelanja produk UMKM Sumatera...
[WA to 081234567890] [Toko UMKM] Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres. Nomor
resi akan muncul di aplikasi dalam 1x24 jam. Terima kasih telah berbelanja produk UMKM
Sumatera Barat di toko kami!
[LOG] Siti Rahma: Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres. Nomor resi akan
muncul di aplikasi dalam 1x24 jam. Terima kasih telah berbelanja produk UMKM Sumatera Barat
di toko kami!
Terkirim ke Siti lewat 4 kanal
[WA to +6281298765432] [Toko UMKM] Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres.
Nomor resi akan muncul di aplikasi dalam 1x24 jam. Terima kasih telah berbelanja produk UMKM
Sumatera Barat di toko kami!
[LOG] Budi: Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres. Nomor resi akan muncul di
aplikasi dalam 1x24 jam. Terima kasih telah berbelanja produk UMKM Sumatera Barat di toko
kami!
Terkirim ke Budi lewat 2 kanal
```

Jawab di laporan:

1. Mengapa ada interface **dan** abstract class, bukan salah satunya saja?
2. Apa akibatnya jika `send(...)` tidak dibuat `final`?
3. Mengapa `NotificationService` menyimpan `List<NotificationChannel>`, bukan `List<AbstractChannel>`?

**Checkpoint T2:** `TugasCheck` T2.1–T2.5 lulus. Contoh output lengkap `TugasCheck` untuk NIM 2401010057. Baris `[SMS to ...]`, `[EMAIL to ...]`, dan `[WA to ...]` di tengahnya berasal dari kanal Anda yang dipanggil saat pengujian; itu normal.

```
=== TUGAS CHECK: Praktikum Abstraction ===
NIM: 2401010057 | Contoh Mahasiswa

-- Parameter NIM
  [LULUS] P.1 Konstanta di StudentInfo sesuai NIM (2%, Rp3500, 157 karakter)

-- T1 - Evolusi API & diamond
  [LULUS] T1.1 PaymentMethod.adminFee(long) adalah default method; e-wallet = 0
  [LULUS] T1.2 Biaya admin kartu kredit & transfer sesuai parameter NIM
  [LULUS] T1.3 grandTotal(...) = total + biaya admin (kartu kredit, keranjang contoh,
Ekspres)
  [LULUS] T1.4 Diamond policy(): kartu kredit menggabungkan kedua kebijakan

-- T2 - Sistem notifikasi
  [LULUS] T2.1 NotificationChannel interface; AbstractChannel abstract dengan send(...)
final
  [LULUS] T2.2 EmailChannel, SmsChannel, WhatsAppChannel adalah subclass AbstractChannel
[SMS to 081234567890]
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx...
  [LULUS] T2.3 SmsChannel memotong pesan panjang tepat 157 karakter berakhiran "..."
[EMAIL to uji@example.com] Halo
  [LULUS] T2.4 EmailChannel menolak kontak tanpa email yang valid
[WA to +6281298765432] [Toko UMKM] Pesanan dikirim
  [LULUS] T2.5 NotificationService.broadcast(...) mengembalikan jumlah kanal yang berhasil

=== HASIL: 10 / 10 pemeriksaan lulus ===
```

### T3 – Code review `BadShop` (bobot 20)

**Skenario.** Anda menjadi reviewer (penelaah kode) di tim toko online. File `src/review/BadShop.java` bisa dikompilasi dan dijalankan, tetapi desainnya bermasalah. Jalankan dulu dan amati keluarannya: ada ongkir yang bernilai 0, alamat pesanan yang berubah sendiri, dan program yang berhenti dengan exception. Output `BadShop` yang benar (memang bermasalah):

```
Total cart: 170000
Ongkir EXPRES 2 kg: 0
ORD-1 -> Jl. Sudirman 10, Bukittinggi
ORD-2 -> Jl. Sudirman 10, Bukittinggi
Harga flash sale: 70000
true | Voucher, remaining balance 70000.0
Exception in thread "main" java.lang.UnsupportedOperationException: Voucher is not shipped
      at review.GiftVoucher.ship(BadShop.java:41)
      at review.BadShop.main(BadShop.java:184)
```

1. Temukan **minimal 5 masalah desain** yang berkaitan dengan materi abstraction (abstract class, interface, default method, `final`, nested class). Masalah tambahan di luar itu dihargai sebagai nilai plus.
2. Untuk setiap masalah, isi tabel berikut di laporan:

| No | Lokasi (class/method) | Masalah | Dampak nyata (bug atau risiko) | Perbaikan yang diusulkan |
|---|---|---|---|---|
| 1 | | | | |

3. Refactor **minimal 3** masalah ke file baru `src/review/fixed/BadShopRefactored.java` (package `review.fixed`). Jangan ubah file `BadShop.java` aslinya. Pastikan versi refactor bisa dikompilasi dan semua gejala yang Anda perbaiki hilang.

Contoh output versi refactor acuan yang memperbaiki semua masalah inti. Versi Anda boleh berbeda; yang penting setiap gejala yang Anda perbaiki hilang, misalnya ongkir Ekspres tidak lagi 0, alamat ORD-1 tidak ikut berubah, dan program tidak lagi crash:

```
Total cart: 170000
Ongkir Ekspres 2 kg: 36000
ORD-1 -> Jl. Sudirman 10, Padang
ORD-2 -> Jl. Sudirman 10, Bukittinggi
Harga flash sale: 70000
true | sisa Rp70000
```

### T4 – Refleksi: kapan memakai apa (bobot 15)

Isi dua tabel keputusan berikut dengan kata-kata sendiri. Setiap baris wajib memuat satu contoh dari proyek toko online Anda.

**Tabel 1 – Abstract class vs interface**

| Pertanyaan desain | Abstract class atau interface? | Alasan | Contoh dari proyek |
|---|---|---|---|
| Subclass berbagi field dan konstruktor | | | |
| Satu class perlu beberapa kemampuan sekaligus | | | |
| Rumus bersama yang tidak boleh diubah subclass | | | |
| Kontrak yang perlu ditambah method baru di kemudian hari | | | |

**Tabel 2 – Jenis nested class**

| Kondisi | Static nested, inner, atau anonymous? | Alasan | Contoh dari proyek |
|---|---|---|---|
| Class pembantu tidak membaca data objek luar | | | |
| Objek harus membaca field objek luarnya | | | |
| Implementasi sekali pakai, tidak perlu nama | | | |

### E.5 Format laporan tugas (PDF)

1. Identitas: nama, NIM, kelas, dan output `tools.NimParams` Anda.
2. Tangkapan layar *error compiler* dari L1, L3, L4, T1.a, dan T1.e, masing-masing dengan penjelasan 1–2 kalimat.
3. Jawaban pertanyaan T1 dan T2, diagram UML T2, tabel T3, dan tabel T4.
4. Tangkapan layar `TugasCheck`, `SelfCheck`, dan keluaran `TugasApp`.

Kumpulkan **`TUGAS_<NIM>_<Nama>.zip`** berisi seluruh **folder proyek beserta laporan PDF** ke aplikasi CODATA LAB dan **push code ke Github Repository )** masing-masing **(<YOUR_REPOSITORY>/Praktikum6 /Tugas)**

# F. Penilaian

Nilai praktikum = **40% Latihan + 60% Tugas**.

### Latihan (100 poin)

| Komponen | Poin | Cara menilai |
|---|---|---|
| SelfCheck | 60 | (jumlah pemeriksaan lulus ÷ 23) × 60 |
| Kerapian dan konvensi kode | 15 | Nama class/method/variabel sesuai konvensi Java, package benar, tanpa error code |
| Validasi dan enkapsulasi | 15 | Field `private`, constructor menolak data tidak sah, uang memakai `long` |
| `ShopApp` berjalan sesuai keluaran contoh | 10 | Katalog, ringkasan, pembayaran, struk, voucher tampil benar |

### Tugas (100 poin)

| Bagian | Poin | Sangat baik | Cukup | Kurang |
|---|---|---|---|---|
| T1 Evolusi API dan diamond | 25 | TugasCheck P.1 dan T1 lulus; jumlah error T1.a dan alasannya tepat, termasuk anonymous class; jawaban diamond benar | Kode lulus, penjelasan kurang lengkap | Kode belum lulus atau penjelasan keliru |
| T2 Sistem notifikasi | 40 | UML konsisten dengan kode; T2.1–T2.5 lulus; `send` final memakai method abstract/hook; ketiga jawaban menunjukkan alasan desain | Kode lulus, UML atau alasan desain lemah | Spesifikasi wajib tidak terpenuhi |
| T3 Code review | 20 | ≥ 5 masalah tepat dengan dampak nyata; ≥ 3 refactor benar dan bisa dikompilasi | 3–4 masalah tepat atau refactor sebagian | < 3 masalah tepat |
| T4 Refleksi | 15 | Semua baris benar, alasan sendiri, contoh dari proyek | Sebagian besar benar, contoh umum | Banyak yang keliru atau kosong |

### Kebijakan keaslian

- **Parameter NIM**: Konstanta di `StudentInfo` harus cocok dengan NIM Anda sendiri.
- **Pretest**: Di awal praktikum berikutnya, dosen/asisten memilih beberapa mahasiswa secara acak untuk menjelaskan kodenya selama 2–3 menit. Mahasiswa yang tidak dapat menjelaskan kodenya sendiri dapat dikenai penyesuaian nilai tugas.
- **Penggunaan AI:** Dilarang menggunakan AI generatif apapun untuk menggenerate kode program dalam mengerjakan latihan dan tugas praktikum.

# G. Referensi


1. Deitel, P. & Deitel, H. *Java How to Program, Early Objects*, ed. 11. Pearson.
2. Slide perkuliahan "Abstraction 1 & 2" (interface inheritance, default method, final class/method, inner class).
3. Oracle, *The Java Tutorials*: "Nested Classes" dan "Anonymous Classes".